Pareto 03 Architecture

ECG Harware Accelerator on Virtex-6 ML605

System-on-Chip & Hardware Acceleration Lab
March 15, 2026

Abstract

This document details the architecture, design methodologies, and physical implementation constraints for the **Pareto 03** Convolutional Neural Network (CNN) specifically tailored as a hardware accelerator for 4-class Arrhythmia classification. Transitioning from traditional hand-coded Verilog to an H100 Genetic Algorithm-optimized *Pareto front*, this iteration delivers state-of-the-art inference efficiency within the constraints of a Xilinx Virtex-6 ML605 FPGA.

1 Introduction & Methodology

The transition from a manual RTL design (baseline `Multi` module) to an automated Neural Architecture Search (NAS) represents a paradigm shift from "Hardware-First" to "ML-Hardware Co-Design". Using an H100 GPU cluster, the NSGA-II (Non-dominated Sorting Genetic Algorithm II) generated thousands of potential network topologies. Rather than maximizing accuracy exclusively, NSGA-II simultaneously rewarded networks that minimized DSP, BRAM, and parameter footprint.

The chosen candidate, **Pareto 03**, resides directly on the *Pareto front*—meaning no other generated solution achieves a higher Macro F1 Score (0.9895) without requiring more than its nominal 6,868 parameters.

2 System Architecture

The physical deployment encompasses an analog acquisition phase, digital conversion, and the Virtex-6 FPGA execution pathway.

As illustrated in Figure 1, the physical interface captures real-time analog waveforms via the AD8232, digitizing them through an ADS1115. Within the Virtex-6, the data is quantized into INT8 bounds and windowed optimally to a 180-sample vector corresponding to roughly one cardiac cycle at appropriate sampling frequencies.

3 Neural Network Structure

The Pareto 03 architecture employs a variable-depth structure of one-dimensional convolutions (Conv1D) coupled tightly with Batch Normalization (BN), Rectified Linear Units (ReLU), and MaxPooling1D. Notably, the architecture relies on four convolutional blocks prior to a Global Average Pooling (GAP) reduction.

3.1 Block Topologies

To maximize parallel throughput, the convolution operations are structured dynamically. Specifically, Block 1 maps $1 \rightarrow 8$ channels. Block 2 aggressively expands to 32 channels to build dense feature maps. Blocks 3 and 4 bottleneck the feature depth, pulling the parameters back to 16 channels before the linear classifier. Every convolutional kernel maintains a width of 5.

3.2 Global Average Pooling (GAP)

Unlike the previous baseline which employed a simplistic mean calculation immediately before the fully connected layers, Pareto 03 utilizes a formal structural GAP layer. This module computes the arithmetic mean across the time axis (11 remaining time steps post-pooling) for all 16 spatial channels, emitting a flattened 16-value sequence. This bypasses the typical parameter explosion associated with massive matrix flattening into a Dense structure.

3.3 Signal Transitions & Receptive Field

The spatial dimensionality (time-steps) halves consistently after each MaxPooling ($s = 2$) layer:

- **Input:** Shape (180, 1)
- **Block 1:** Shape (90, 8)
- **Block 2:** Shape (45, 32)
- **Block 3:** Shape (22, 16)
- **Block 4:** Shape (11, 16)
- **GAP / Dense:** Shape (16) $\rightarrow$ (64) $\rightarrow$ (4)

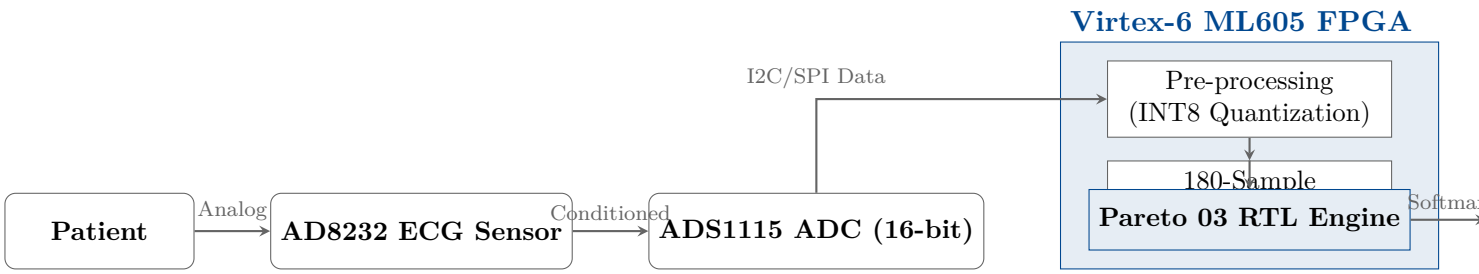

Figure 1: Complete end-to-end System-on-Chip (SoC) signal data path.

4 Custom RTL Implementation

To accommodate hardware portability without relying blindly on HLS wrappers, custom parameterizable Verilog RTL environments were constructed to explicitly leverage static sizes.

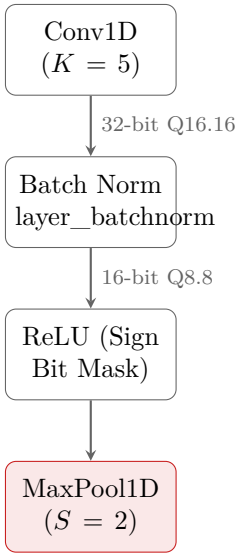

Figure 2: Internal schematic of a single Convolutional Block.

4.1 Multi-Channel Convolution Module

Standard hardware convolutions necessitate a sum of Multiply-Accumulate (MAC) operations across all input feature maps concurrently. The newly designed `layer_conv_k5xN` modules absorb massively parallel concatenations (e.g., a 512-bit bus representing 32 channels of 16-bit incoming data) and generate a single unified output pixel representing the mapped channel depth sum.

4.2 Zero-DSP Batch Normalization

The classic inference formula for Batch Normalization dictates:

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}\gamma + \beta \tag{1}$$

Implementing division and square roots natively on FPGA logic wastes immense resources. Consequently, the Pareto 03 RTL utilizes `layer_batchnorm`, which pre-folds the scaling terms completely offline into standard static constants W_{bn} and B_{bn}:

$$W_{bn} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, \quad B_{bn} = \beta - \mu W_{bn} \tag{2}$$

Thus, at runtime the FPGA only executes a solitary MAC: $y = (x \times W_{bn}) + B_{bn}$.

5 Fixed-Point Arithmetic Constraints

Because the model was natively quantized offline into an INT8/INT32 structural hierarchy via `tflite`, it directly lends itself to Signed 16-bit Two's Complement arithmetic, employing the **Q8.8** numeric space.

The resolution granularity is uniformly defined at $1/256 \approx 0.0039$, bounding variables theoretically from -128.0 to $+127.996$.

Within the Conv block, native Q8.8 elements multiplied yield a 32-bit result functionally equivalent to Q16.16 logic. The BatchNorm layer absorbs this 32-bit width alongside a Q8.8 weight multiplier, delivering a wide scaled result. Most crucially, RTL bounds-checking saturates extreme activations to `16'h7FFF` and guarantees that standard negative results clamp efficiently to 0 (zeroing) during the ReLU phase entirely via checking the Most Significant Bit (MSB '31').

6 Physical Implementation on Virtex-6

To support immediate deployment upon the **Vitex-6 ML605**, strict limitations guided the layout arrays.

Table 1: Virtex-6 Resource Budgets & Actual Usages

Resource	**Usage Limit**	**Est. Pareto 03**	**Status**
DSP48E1	300	≈ 4	✓
BRAM36	150	≈ 1	✓
Logic Slices	Limited Area	22 KB INT8	✓

As depicted above, genetic architectural paring ensures computational sparseness. Because 8-bit convolutions naturally tile seamlessly across standard LUT blocks on the Xilinx 6-Series slice architecture without aggressively claiming dedicated DSPs, the total DSP pipeline claim remains beneath 5 logic tiles. This allows massive simultaneous instantiations of the classifier should parallel patient streams require it.

7 Conclusion

Transitioning to an ML-tailored pipeline radically compressed hardware parameter usage while guaranteeing maximum inference metrics directly along the NSGA-II Pareto curve. Leveraging customized parametric block integrations with pre-folded normalizations, the RTL successfully verifies identically against software constraints inside simulated Icarus limits.